EdgeX: Perpetual Contract

Security Review

Cantina Managed review by:
Kurt Barry, Lead Security Researcher
Cccz, Security Researcher

March 9, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **5**
3.1 High Risk . . . 5
3.1.1 Attackers can settle Twap orders at any price . . . 5
3.1.2 Incorrect checking of `FundingIndex` . . . 7
3.1.3 Users can generate bad debts to profit . . . 8
3.2 Medium Risk . . . 9
3.2.1 Deposit lacks signature verification . . . 9
3.2.2 Deleveraging lacks signature verification from privileged users . . . 11
3.2.3 Incorrect calculation in `getMaxRemovableIsolatedMargin()` . . . 11
3.2.4 The strict check in deleverage make it near unavailable . . . 12
3.2.5 Precision loss in oracle prices . . . 13
3.2.6 Incorrect rounding direction for isolated position funding fees . . . 14
3.2.7 Malicious users can trade dust tokens at near-zero cost . . . 15
3.2.8 Liquidation has no price limit . . . 17
3.3 Low Risk . . . 18
3.3.1 `depositWithPermit()` allows attackers to deposit funds that user has approved into the `edgexVault` . . . 18
3.3.2 Position Information Can Be Overwritten Resulting in Corrupted State . . . 19
3.3.3 Additional Verification Check Recommendations . . . 19
3.3.4 Positions Can Exceed Value and Risk Bounds In Extreme Scenarios, Blocking Operations 20
3.3.5 Rounding direction in DelistSettle is incorrect . . . 21
3.3.6 Withdrawal structure lacks timestamps . . . 21
3.3.7 Account status has no buffer between healthy and liquidatable . . . 21
3.4 Gas Optimization . . . 22
3.4.1 Gas Optimizations . . . 22
3.5 Informational . . . 23
3.5.1 Minor Issues: Redundancies, Event Issues, Code Clarity, etc.. . . . 23
3.5.2 EIP-712 Compliance . . . 26

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jan 15th to Jan 29th the Cantina team conducted a review of edgex-perpetual-contract on commit hash 09dccf06. The team identified a total of **21** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	3	2	1
Medium Risk	8	5	3
Low Risk	7	5	2
Gas Optimizations	1	1	0
Informational	2	2	0
Total	**21**	**15**	**6**

2.1 Scope

The security review had the following components in scope for edgex-perpetual-contract on commit hash 09dccf06:

```
src
├── Constants.sol
├── MultiTransactionProcessor.sol
├── TransactionProcessor.sol
├── account
│   ├── AccountManager.sol
│   ├── IAccountManager.sol
│   └── RegisterAccount.sol
├── config
│   └── GeneralConfig.sol
├── deploy
│   └── Create2Factory.sol
├── deposit
│   ├── IEdgexVault.sol
│   ├── ILoadingZone.sol
│   └── LoadingZone.sol
├── funding
│   ├── Funding.sol
│   ├── FundingLib.sol
│   └── IFunding.sol
├── oracle
│   ├── IOracle.sol
│   └── Oracle.sol
├── order
│   ├── IOrderManager.sol
│   ├── ITwapOrder.sol
│   ├── LimitOrder.sol
│   ├── LimitOrderLib.sol
│   ├── OrderManager.sol
│   ├── TwapOrder.sol
│   └── TwapOrderManager.sol
├── position
│   ├── IPositionManager.sol
│   ├── PositionAssetLib.sol
│   └── PositionManager.sol
├── signer
│   ├── ISignerManager.sol
│   ├── SignerManager.sol
```

```
│   └── SignerPermissions.sol
├── transactions
│   ├── DeleverageTransLib.sol
│   ├── DelistSettleTransLib.sol
│   ├── DepositTransLib.sol
│   ├── FundingTickTransLib.sol
│   ├── IsolatedMarginTransLib.sol
│   ├── LiquidateTransLib.sol
│   ├── OraclePriceTickTransLib.sol
│   ├── PermissionsTransLib.sol
│   ├── RegisterAccountTransLib.sol
│   ├── SetMarginPreferenceTransLib.sol
│   ├── TradeTransLib.sol
│   ├── TransferTransLib.sol
│   ├── TwapTransLib.sol
│   └── WithdrawalTransLib.sol
└── utils
    ├── EIP712Lib.sol
    ├── EIP712VerificationLib.sol
    ├── SignaturesLib.sol
    └── Uint64Set.sol
```

3 Findings

3.1 High Risk

3.1.1 Attackers can settle Twap orders at any price

Severity: High Risk

Context: LimitOrderLib.sol#L207-L236

Description: When settling Twap orders, sub-orders are required to have the same basic data as the Twap order, and no signature verification is performed.

```
if (isTwapSubOrder(self)) {
    // TWAP sub-order validation with security guardrails
    require(
        address(twapOrderManager) != address(0), "LimitOrderLib: TWAP order manager
        ↪  required for sub-orders"
    );

    // Get parent order for binding validation
    TwapOrder memory parent = twapOrderManager.getParentOrder(self.parentOrderHash);
    require(parent.orderHash != bytes32(0), "LimitOrderLib: parent order not found");

    // Critical security validations
    require(self.base.signer == parent.base.signer, "LimitOrderLib: signer mismatch");
    require(self.base.accountId == parent.base.accountId, "LimitOrderLib: account
    ↪  mismatch");
    require(self.assetIdSynthetic == parent.assetIdSynthetic, "LimitOrderLib: synthetic
    ↪  asset mismatch");
    require(self.assetIdCollateral == parent.assetIdCollateral, "LimitOrderLib:
    ↪  collateral asset mismatch");
    require(self.isBuyingSynthetic == parent.isBuyingSynthetic, "LimitOrderLib: direction
    ↪  mismatch");

    // Index bounds validation
    require(
        self.subOrderIndex >= 0 && self.subOrderIndex < parent.subOrderCount,
        "LimitOrderLib: invalid sub-order index"
    );

    require(block.timestamp <= parent.base.expirationTimestamp, "LimitOrderLib: parent
    ↪  order expired");

    // Enforce parent-level fulfillment cap (critical security)
    twapOrderManager.updateParentFulfillment(self.parentOrderHash, updateAmount);

    // No signature verification needed - inherits from parent
```

The problem here is that Twap orders lack price data, which allows attackers to settle Twap orders at any price.

Proof of Concept:

```
function testTwapOrderTradeNoSignature() public {
    // Create TWAP order
    TwapOrder memory twapOrder = _createSampleTwapOrder();

    vm.prank(address(transactionProcessor));
    twapOrderManager.createTwapOrder(twapOrder);

    // Verify TWAP order was created
    TwapOrder memory storedTwapOrder =
    ↪  twapOrderManager.getParentOrder(twapOrder.orderHash);
    assertEq(storedTwapOrder.orderHash, twapOrder.orderHash);
    assertEq(storedTwapOrder.totalSize, 2);
    assertEq(storedTwapOrder.subOrderCount, 2);
```

```
// Create trades with TWAP order's sub-order in aOrder
uint256 orderNonce = 1;
vm.startPrank(address(multiTransactionProcessor));
OrderBase memory aBase = OrderBase({
    nonce: orderNonce, signer: deployer, accountId: deployerUid,
    ↪  expirationTimestamp: 1, signature: "0x"
});
LimitOrder memory aOrder = LimitOrder({
    base: aBase,
    amountSynthetic: 1,
    amountCollateral: 100,
    amountFee: 1,
    assetIdSynthetic: synthAssetId,
    assetIdCollateral: 1,
    isBuyingSynthetic: true,
    orderHash: "0x",
    parentOrderHash: twapOrder.orderHash,
    subOrderIndex: 0
});
aOrder.orderHash = aOrder.getOrderHash(address(transactionProcessor));

// no signature
// (uint8 v, bytes32 r, bytes32 s) = vm.sign(PRIVATE_KEY, aOrder.orderHash);
// aOrder.base.signature = abi.encodePacked(r, s, v);

OrderBase memory bBase = OrderBase({
    nonce: orderNonce, signer: bOwner, accountId: bOwnerUid, expirationTimestamp: 1,
    ↪  signature: "0x"
});
LimitOrder memory bOrder = LimitOrder({
    base: bBase,
    amountSynthetic: 1,
    amountCollateral: 100,
    amountFee: 1,
    assetIdSynthetic: synthAssetId,
    assetIdCollateral: 1,
    isBuyingSynthetic: false,
    orderHash: "0x",
    parentOrderHash: bytes32(0),
    subOrderIndex: 0
});
bOrder.orderHash = bOrder.getOrderHash(address(transactionProcessor));
// Sign message using private key
(uint8 bV, bytes32 bR, bytes32 bS) = vm.sign(PRIVATE_KEY_B, bOrder.orderHash);
bOrder.base.signature = abi.encodePacked(bR, bS, bV);

Trade memory trade = Trade({
    partyAOrder: aOrder,
    partyBOrder: bOrder,
    actualCollateral: 100,
    actualSynthetic: 1,
    actualAFee: 1,
    actualBFee: 0
});
bytes memory message = abi.encode(trade);

bytes memory txBytes = getMessage(TYPE_TRADE, message);

bytes[] memory txs = new bytes[](1);
txs[0] = txBytes;
vm.stopPrank();
vm.startPrank(deployer);
uint64 configVersion = generalConfig.configVersion();
bytes32 lastBatchHash = multiTransactionProcessor.lastBatchHash();
```

```
    multiTransactionProcessor.processMultiTransactions(txs, configVersion,
    ↪  lastBatchHash);
    vm.stopPrank();
    vm.startPrank(address(multiTransactionProcessor));

    AccountPosition memory accountPosition =
    ↪  positionManager.getAccountPositionByUID(deployerUid);
    assertEq(accountPosition.collateralBalance, 999899);
    assertEq(accountPosition.assets[0].balance, 1);

    AccountPosition memory accountPositionB =
    ↪  positionManager.getAccountPositionByUID(bOwnerUid);
    assertEq(accountPositionB.collateralBalance, 100);
    assertEq(accountPositionB.assets[0].balance, -1);
}
```

Recommendation: It is recommended to add price data to Twap orders and require sub-orders to match the price of the Twap order based on the trade direction.

EdgeX: Acknowledged. TWAP sub-orders are intentionally not user-signed. Users cannot predict future prices when signing, so price constraints must be enforced off-chain by the Sequencer.This follows the same trust model as the rest of the system - users trust the Sequencer for fair execution. Price fairness is guaranteed through oracle validation and off-chain matching engine controls, not on-chain signatures.

Cantina Managed: Acknowledged.

3.1.2 Incorrect checking of `FundingIndex`

Severity: High Risk

Context: FundingLib.sol#L28

Description: When setting the funding index, `isIndexInRange()` is checking the cumulative index rather than the incremental index. Since the cumulative index is market-dependent, it continues to increase when there are more long positions than short positions, and it continues to decrease when there are fewer long positions than short positions, so the cumulative index should not have a specific range, instead, the range should be limited for the incremental index.

```
function isIndexInRange(uint64 assetId, int256 cumulativeIndex, GeneralConfig
↪  generalConfig)
    public
    view
    returns (bool)
{
    int256 maxFundingRate = generalConfig.getMaxFundingRate(assetId);
    if (maxFundingRate == 0) {
        return true;
    }
    return cumulativeIndex >= -maxFundingRate && cumulativeIndex <= maxFundingRate;
}

function process(AssetFundingSnapshot[] memory newFundingSnapshots, IFunding funding,
↪  GeneralConfig generalConfig)
    public
{
    uint256 len = newFundingSnapshots.length;
    require(len > 0, "FundingLib: no funding snapshots");
    for (uint256 i = 0; i < len;) {
        AssetFundingSnapshot memory newFundingSnapshot = newFundingSnapshots[i];
        if (!isIndexInRange(newFundingSnapshot.assetId,
        ↪  newFundingSnapshot.snapshot.cumulativeIndex, generalConfig))
        {
            revert("FundingLib: funding index is out of range");
        }

        funding.updateFundingIndex(newFundingSnapshot);
```

```
        unchecked {
            ++i;
        }
    }
}
```

Recommendation:

```
  function process(AssetFundingSnapshot[] memory newFundingSnapshots, IFunding funding,
  ↪  GeneralConfig generalConfig)
      public
  {
      uint256 len = newFundingSnapshots.length;
      require(len > 0, "FundingLib: no funding snapshots");
      for (uint256 i = 0; i < len;) {
          AssetFundingSnapshot memory newFundingSnapshot = newFundingSnapshots[i];
-         if (!isIndexInRange(newFundingSnapshot.assetId,
↪  newFundingSnapshot.snapshot.cumulativeIndex, generalConfig))
+         (int256 globalIndex,) = this.getFundingIndex(newFundingSnapshot.assetId); //
↪  returns (0, false) on first setting.
+         int256 incrementalIndex = newFundingSnapshot.snapshot.cumulativeIndex -
↪  globalIndex;
+         if (!isIndexInRange(newFundingSnapshot.assetId, incrementalIndex,
↪  generalConfig))
          {
              revert("FundingLib: funding index is out of range");
          }

          funding.updateFundingIndex(newFundingSnapshot);
          unchecked {
              ++i;
          }
      }
  }
```

EdgeX: Fixed in commit 659fe3d.

Cantina Managed: Verified.

3.1.3 Users can generate bad debts to profit

Severity: High Risk

Context: PositionManager.sol#L292-L294

Description: In `_checkValidTransition()`, an account that is at risk but has no debt will pass the check.

This creates an opportunity for malicious users to generate bad debt.

```
function _checkValidTransition(
    AccountPosition memory initPosition,
    AccountPosition memory updatedPosition,
    bool isLiquidation
) internal view {
    (int256 updatedTotalValue, int256 updatedTotalRisk) =
    ↪  getAccountPositionStatus(updatedPosition);
    if (updatedTotalRisk <= updatedTotalValue * FXP_32_ONE) {
        // Position is healthy.
        // For liquidation: check sign change to prevent over-closing.
        // For normal trades: no restriction, user can flip position direction.
        if (isLiquidation) {
            _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, true);
        }
        return;
    }
    if (updatedTotalRisk == 0) {
```

```
        return;
    }
```

Consider ETH/USDT = 3000, risk factor = 0.1.

The user's account collateral is 3000 USDT, and the user uses 30000 USDT to trade 10 ETH. The collateral is -24000 USDT, and the synthetic asset is 10 ETH.

The user then sells 10 ETH for 1 USDT. The collateral is -23999 USDT, and the synthetic asset is 0 ETH. The account is at risk, but because the debt is 0, it passes `_checkValidTransition()`.

The counterparty buys 10 ETH for 1 USDT and profits, leaving the protocol with a bad debt of -23999 USDT.

Proof of Concept: The following PoC shows a user buying 100 BTC at price 50000, then selling 100 BTC at price 1 (dust). The trade succeeds because `updatedTotalRisk == 0`, the counterparty will profit from buying BTC at the low price, and leaving bad debt.

```
function testPoC() public {
    bytes[] memory txs = new bytes[](1);

    txs[0] = createTradeTransaction(
        user1Uid,
        mm1Uid,
        assetId1,
        100,
        100 * 50000,
        0, // User1 buys 1 BTC for 50k collateral
        PRIVATE_KEY_USER1,
        PRIVATE_KEY_MM1
    );

    executeMultiTransactions(txs);
    verifyPosition(user1Uid, "User1");

    txs[0] = createTradeTransaction(
        mm1Uid,
        user1Uid,
        assetId1,
        100,
        1,
        0, // User1 Sell 2 BTC for 100k collateral
        PRIVATE_KEY_MM1,
        PRIVATE_KEY_USER1
    );
    executeMultiTransactions(txs);
    verifyPosition(user1Uid, "User1");
}
```

Recommendation: It is recommended to revert directly when the account is at risk and `updatedTotalRisk == 0`, as the protocol has sufficient liquidation buffer, making bad debt cases nearly impossible.

EdgeX: Fixed in commit 46385b8.

Cantina Managed: Verified.

3.2 Medium Risk

3.2.1 Deposit lacks signature verification

Severity: Medium Risk

Context: DepositTransLib.sol#L10-L13

Description: The Deposit structure lacks the `signature field`, and the processing lacks signature verification.

```
struct Deposit {
    uint64 accountId; // Account ID which deposited
    int256 amount; // Deposit amount
}
//...
library DepositTransLib {
    function process(
        Deposit memory self,
        IPositionManager position,
        IAccountManager accountManager,
        ILoadingZone loadingZone
    ) public {
        require(self.amount > 0, "DepositTransLib: amount must be greater than 0");
        require(self.accountId != 0, "DepositTransLib: accountId is 0");
        require(accountManager.isAccountIdRegistered(self.accountId), "DepositTransLib:
        ↪  accountId is not registered");

        // Process deposit from LoadingZone (move from user balance to system balance)
        require(
            loadingZone.processDeposit(accountManager.getAddressByAccountId(self.account
            ↪  Id), uint256(self.amount)),
            "DepositTransLib: LoadingZone processing failed"
        );

        // Update user's position with the deposited amount (existing logic)
        position.updatePositionInDict(self.accountId, self.amount,
        ↪  NO_SYNTHETIC_DELTA_ASSET_ID, 0);
    }
}
```

While the similar Withdrawal has the signature field and signature verification. `verifyWithdrawalSignature()` will verifie that the signer signed the entire structure and then check if the signer has the account's permissions.

```
library WithdrawalTransLib {
    function process(
        Withdrawal memory self,
        IPositionManager position,
        IAccountManager accountManager,
        ILoadingZone loadingZone,
        ISignerManager signerManager
    ) public {
        require(self.amount > 0, "WithdrawalTransLib: amount must be greater than 0");
        require(self.from != 0, "WithdrawalTransLib: withdrawal account ID is 0");
        require(self.toAddress != address(0), "WithdrawalTransLib: to address is 0");
        address fromAddress = accountManager.getAddressByAccountId(uint64(self.from));
        require(fromAddress != address(0), "WithdrawalTransLib: withdrawal address is not
        ↪  registered");
        // Basic collateral balance check
        if (position.getCollateralBalanceByAccountId(uint64(self.from)) < self.amount) {
            revert("WithdrawalTransLib: insufficient balance");
        }

        // Only EIP-712 signature verification
        require(
            verifyEIP712Signature(self, signerManager, address(this)),
            "WithdrawalTransLib: invalid EIP-712 signature or permissions"
        );
```

Ignoring front-end checks, the lack of signature verification in the Deposit could potentially allow attackers to update the account's position margin.

Recommendation: It is recommended to add signature verification to the Deposit operation.

EdgeX: Acknowledged. We consider this behavior intentional:LoadingZone = User Authorization: When

users deposit into LoadingZone, they explicitly consent to enter the EdgeX system. The deposit itself is the authorization.Equivalent Trust Model: Users already trust the Sequencer/Batcher to execute transactions. Trusting them to process deposits from LoadingZone is not an additional trust assumption.No Additional Risk: Funds in LoadingZone are already committed to the trading system. Processing them as deposits does not introduce new attack vectors.Conclusion: Requiring user signatures for deposit would add complexity without meaningful security benefit, as the trust boundary is crossed when funds enter LoadingZone, not when they are processed into the trading account.

Cantina Managed: Acknowledged.

3.2.2 Deleveraging lacks signature verification from privileged users

Severity: Medium Risk

Context: DeleverageTransLib.sol#L27-L32

Description: According to the comments, the deleverage operation does not require the signature of the deleverager.

Considering this is a sensitive operation for the deleverager's position and a forced safety operation, even if the deleverager's signature is not required, it should still require the signature of a privileged user, similar to requiring the fundingFeeder's signature when setting the funding rate.

```
 *  - Authorization: we only require that both party addresses are registered signers for
 *    their accounts. There is no end-user EIP-712 signature for deleverage itself
 ↪  because
 *    this is a forced safety action triggered when the protocol detects insolvency risk.
 *  - Safety checks: verify tradability, signer authorization, riskiness, and symmetric
 *    collateral/synthetic deltas at oracle PX (enforcing fairness). If any check fails,
 *    the transaction reverts and state is unchanged.
 * Example: Carol's health < 1.0. Sequencer selects Dave (willing to take the other side)
 * and submits a deleverage filling amountSynthetic X at oracle price P, so Carol's
 ↪  collateral
 * increases by X*P and synthetic decreases by X. If Carol is not risky, or Dave/Carol
 ↪  signer
 * isn't authorized, or the amount doesn't match oracle PX, this function reverts.
 */
function process(
    Deleverage memory self,
    IPositionManager position,
    GeneralConfig generalConfig,
    IAccountManager accountManager,
    IOracle oracle
) public {
```

Recommendation: It is recommended to add signature verification from privileged users to the deleveraging process.

EdgeX: Fixed in commit 1d5ebf4.

Cantina Managed: Verified.

3.2.3 Incorrect calculation in `getMaxRemovableIsolatedMargin()`

Severity: Medium Risk

Context: PositionManager.sol#L668-L672

Description: In `getMaxRemovableIsolatedMargin()`, `mmrRep` and `equityRep` are scaled differently, with `mmrRep` being much larger than `equityRep`. This means that when an account has debt, `maxRemovable` will be 0, the user cannot withdraw excess margin.

```
function getMaxRemovableIsolatedMargin(uint64 accountId, uint64 assetId) public view
↪  returns (int256) {
    require(accountId != 0, "PM: accountId 0");
    require(assetId != 0 && assetId != NO_SYNTHETIC_DELTA_ASSET_ID, "PM: assetId
    ↪  invalid");
```

```
    PositionAsset memory asset = syntheticPositions[_syntheticPositionKey(accountId,
    ↪  assetId)];
    require(asset.packed.getAssetId() != 0, "PM: pos not found");
    require(
        _accountAssetMarginPref[_syntheticPositionKey(accountId, assetId)] ==
        ↪  MARGIN_MODE_ISOLATED,
        "PositionManager: not isolated"
    );

    int256 oraclePrice = oracle.getPrice(assetId);
    int256 absBalance = asset.balance < 0 ? -asset.balance : asset.balance;
    (int256 riskFactor, bool exist) = generalConfig.getRiskFactor(assetId, absBalance,
    ↪  oraclePrice);
    require(exist, "PM: no risk");

    int256 valueRep = asset.balance * oraclePrice;
    int256 absValueRep = absBalance * oraclePrice;
    int256 mmrRep = absValueRep * riskFactor;
    int256 equityRep = (asset.isolatedCollateral * FXP_32_ONE) + valueRep;
    if (equityRep <= mmrRep) {
        return 0;
    }
    int256 maxRemovable = (equityRep - mmrRep) / FXP_32_ONE;
    if (maxRemovable > asset.isolatedCollateral) {
        maxRemovable = asset.isolatedCollateral;
    }
```

Recommendation:

```
  int256 valueRep = asset.balance * oraclePrice;
  int256 absValueRep = absBalance * oraclePrice;
- int256 mmrRep = absValueRep * riskFactor;
+ int256 mmrRep = (absValueRep * riskFactor + FXP_32_ONE - 1) / FXP_32_ONE; // rounding
 ↪  up
  int256 equityRep = (asset.isolatedCollateral * FXP_32_ONE) + valueRep;
  if (equityRep <= mmrRep) {
      return 0;
  }
  int256 maxRemovable = (equityRep - mmrRep) / FXP_32_ONE;
```

EdgeX: Fixed in commit 989d38a.

Cantina Managed: Verified.

3.2.4 The strict check in deleverage make it near unavailable

Severity: Medium Risk

Context: DeleverageTransLib.sol#L53

Description: In deleveraging, the strict check requires

```
amountSynthetic * oraclePrice == amountCollateral * FXP_32_ONE
```

Since `oraclePrice` is uncontrollable, the way to make the equation true is to make `amountCollateral = x * oraclePrice` and `amountSynthetic = FXP_32_ONE * x`. If the balance of the liquidated account is less than `oraclePrice`, this requires x to be less than 1, which means that `x * oraclePrice` and `FXP_32_ONE * x` do not undergo any rounding.

```
function process(
    Deleverage memory self,
    IPositionManager position,
    GeneralConfig generalConfig,
    IAccountManager accountManager,
    IOracle oracle
) public {
```

```
    if (self.amountSynthetic <= 0 || self.amountCollateral <= 0) {
        revert("DeleverageTransLib: amountSynthetic or amountCollateral is less than or
        ↪  equal to 0");
    }
    require(generalConfig.isTradable(self.syntheticAssetId), "DeleverageTransLib:
    ↪  syntheticAssetId is not tradable");
    require(!generalConfig.isDelisted(self.syntheticAssetId), "DeleverageTransLib:
    ↪  syntheticAssetId is delisted");
    int256 oraclePriceValue = oracle.getPrice(self.syntheticAssetId);
    int256 scaledActualCollateral = self.amountCollateral * FXP_32_ONE;
    int256 expectedCollateral = self.amountSynthetic * oraclePriceValue;
    require(scaledActualCollateral == expectedCollateral, "DeleverageTransLib:
    ↪  collateral/synthetic ratio mismatch");
```

Considering the BTC price is 90,000 USDT,

```
oraclePrice = 90000e18 * 1e6 * (1 << 32) / (1e8 * 1e18) = 3865470566399
```

If the liquidated user's debt value is less than 3865470e6 USDT, the selected `x` must be the common divisor of 3865470566399 and FXP_32_ONE. This makes it near unavailable.

Recommendation: It is recommended to allow slippage between `scaledActualCollateral` and `expectedCollateral`.

EdgeX: Fixed in commit 1d5ebf4.

Cantina Managed: Verified.

3.2.5 Precision loss in oracle prices

Severity: Medium Risk

Context: OraclePriceTickTransLib.sol#L82-L92

Description: When formatting prices, the protocol accounts for the asset's resolution. When the asset has 18 decimals, precision loss can cause large price deviations.

Considering ETH price as `3000e18`, USDT resolution as `1e6`, and ETH resolution as `1e18`,

```
convertedPrice = 3000e18 * 1e6 * (1 << 32) / (1e18 * 1e18) = 12.88
```

rounded to 13. And the value of 1e18 ETH would be (in USDT):

```
1e18 * 13 / (1<<32) == 3026.798367e6
```

For other tokens with smaller values but resolution of 18, the value deviation caused by precision loss will be even greater.

```
function _convertPriceToQ32_32(OraclePrice memory price, uint64 assetId, GeneralConfig
↪  generalConfig)
    private
    view
    returns (OraclePrice memory)
{
    SyntheticAssetInfo memory assetInfo = generalConfig.getSyntheticAssetInfo(assetId);
    CollateralAssetInfo memory collateralInfo = generalConfig.getCollateralAssetInfo();

    // Only convert if both resolutions are available
    if (assetInfo.resolution > 0 && collateralInfo.resolution > 0) {
        // numerator = external_price * collateral_resolution * FXP_32_ONE
        int256 numerator = price.price * collateralInfo.resolution * FXP_32_ONE;
        // denominator = synthetic_resolution * EXTERNAL_PRICE_FIXED_POINT_UNIT
        int256 denominator = assetInfo.resolution * EXTERNAL_PRICE_FIXED_POINT_UNIT;
        // Add denominator/2 for rounding
        int256 convertedPrice = (numerator + denominator / 2) / denominator;
        require(convertedPrice > 0, "OraclePriceTickTransLib: converted price zero");
```

```
        return OraclePrice({price: convertedPrice, timestamp: price.timestamp});
    }

    // If resolutions not available, return original price
    return price;
}
```

Recommendation: It is recommended to increase the precision of prices.

EdgeX: Acknowledged. The analysis assumes synthetic_resolution = `1e18` (raw ETH decimals). However, in our actual deployment, synthetic assets like ETHUSDC use resolution = `1e10`, not `1e18`. With resolution = `1e10`:

- `convertedPrice` ≈ `1,288,490,189` (a 10-digit integer).
- Precision loss: < 0.0000001% (less than one billionth).

This level of precision loss is negligible and well within acceptable tolerance for a trading system. The issue only becomes significant with resolution = `1e18`, which is not used in our production configuration.

Cantina Managed: Acknowledged.

3.2.6 Incorrect rounding direction for isolated position funding fees

Severity: Medium Risk

Context: Funding.sol#L69-L73

Description: The rounding direction for funding fees is inconsistent:

1. Funding fees for cross positions are rounded up:

   [] (position, collateralFXP) = _applyPositionInner(position); position.collateralBalance = collateralFXP / FXP_32_ONE;

2. While funding fees for isolated positions are rounded down:

   [] if (position.assets[i].packed.getMarginMode() == MARGIN_MODE_ISOLATED) { position.assets[i].isolatedCollateral -= fxpDeltaFunding / FXP_32_ONE;

This is because when applying funding fees to isolated positions, the calculation first divides and then subtracts. Consider `deltaFundingIndex = FXP_32_ONE - 1` (Rounding present).

- For Long isolated position 1, balance is 2, `isolatedCollateral` starts at 5. After applying funding fees

  [] isolatedCollateral = 5 - (2 * FXP_32_ONE - 2) / FXP_32_ONE = 4

- For Long isolated position 2, balance is 2, isolatedCollateral starts at 5. After applying funding fees

  [] isolatedCollateral = 5 - (2 * FXP_32_ONE - 2) / FXP_32_ONE = 4

- For Short isolated position 1, balance is -4, isolatedCollateral starts at 5. After applying funding fees

  [] isolatedCollateral = 5 - (- 4FXP_32_ONE + 4) / FXP_32_ONE = 8

Users pay a total of 2 funding fees, but receive a total of 3 funding fees, making the protocol insolvent.

Recommendation: It is recommended to make their calculations consistent.

```
  if (position.assets[i].packed.getMarginMode() == MARGIN_MODE_ISOLATED) {
-     position.assets[i].isolatedCollateral -= fxpDeltaFunding / FXP_32_ONE;
+     position.assets[i].isolatedCollateral = (position.assets[i].isolatedCollateral *
↪  FXP_32_ONE - fxpDeltaFunding) / FXP_32_ONE
  } else {
```

EdgeX: Fixed in commit 912bdcd.

Cantina Managed: Verified.

3.2.7 Malicious users can trade dust tokens at near-zero cost

Severity: Medium Risk

Context: LimitOrderLib.sol#L104-L131

Description: For flexibility, the two roundings in `validateLimitOrderFairness()` result in the actual price being slightly worse than users might expect.

For cases where `isBuyingSynthetic == true`, the ideal condition requires

```
actualCollateral / actualSynthetic < amountCollateral / amountSynthetic
```

This means users can trade the same amount of Synthetic with less Collateral. The condition

```
(actualCollateral - 1) / actualSynthetic < amountCollateral / amountSynthetic
```

implies that the trade is also valid when spending more Collateral.

For cases where `isBuyingSynthetic == false`, the ideal condition requires

```
actualSynthetic / actualCollateral < amountSynthetic / amountCollateral
```

This means users can trade more Collateral for the same amount of Synthetic. The condition

```
actualSynthetic / (actualCollateral + 1) < amountSynthetic / amountCollateral
```

implies that the trade is also valid when receiving less Collateral.

```
if (self.isBuyingSynthetic) {
    int256 actualSold = actualCollateral;
    int256 actualBought = actualSynthetic;
    int256 amountSell = amountCollateral;
    int256 amountBuy = self.amountSynthetic;
    if (actualSold == 0) {
        return;
    }

    // Since we sell collateral, we round actual_sold.
    // (actual_sold - 1) / actual_bought < amount_sell / amount_buy, thus
    // (actual_sold - 1) * amount_buy < amount_sell * actual_bought.
    require(
        (actualSold - 1) * amountBuy < amountSell * actualBought,
        "LimitOrderLib: (actualSold - 1) * amountBuy >= amountSell * actualBought"
    );
} else {
    int256 actualSold = actualSynthetic;
    int256 actualBought = actualCollateral;
    int256 amountSell = self.amountSynthetic;
    int256 amountBuy = amountCollateral;
    // Since we buy collateral, we round actual_bought.
    // actual_sold / (actual_bought + 1) < amount_sell / amount_buy, thus
    // actual_sold * amount_buy < amount_sell * (actual_bought + 1).
    require(
        actualSold * amountBuy < amountSell * (actualBought + 1),
        "LimitOrderLib: actualSold * amountBuy >= amountSell * actualBought"
    );
```

The problem here is that for dust trades, the rounding greatly reduces the actual price. For example, when `actualSold == 1`, the left side is 0, and the right side can be any non-zero value, meaning the trade can be executed at any price.

Proof of Concept: The PoC below shows that Order A wants to buy 1000 synthetic assets with 1 collateral, and Order B wants to sell 1 synthetic assets with 1 collateral. The price clearly doesn't meet Order A's requirements, but the trade is successful.

```
function testOverTrade() public {
    uint256 orderNonce = 1;
```

```
OrderBase memory aBase = OrderBase({
    nonce: orderNonce, signer: deployer, accountId: deployerUid,
    ↪  expirationTimestamp: 1, signature: "0x"
});
LimitOrder memory aOrder = LimitOrder({
    base: aBase,
    amountSynthetic: 1000,
    amountCollateral: 1,
    amountFee: 0,
    assetIdSynthetic: synthAssetId,
    assetIdCollateral: 1,
    isBuyingSynthetic: true,
    orderHash: "0x",
    parentOrderHash: bytes32(0),
    subOrderIndex: 0
});
aOrder.orderHash = aOrder.getOrderHash(address(transactionProcessor));
// Sign message using private key
(uint8 v, bytes32 r, bytes32 s) = vm.sign(PRIVATE_KEY, aOrder.orderHash);
aOrder.base.signature = abi.encodePacked(r, s, v);

OrderBase memory bBase = OrderBase({
    nonce: orderNonce, signer: bOwner, accountId: bOwnerUid, expirationTimestamp: 1,
    ↪  signature: "0x"
});
LimitOrder memory bOrder = LimitOrder({
    base: bBase,
    amountSynthetic: 1,
    amountCollateral: 1,
    amountFee: 0,
    assetIdSynthetic: synthAssetId,
    assetIdCollateral: 1,
    isBuyingSynthetic: false,
    orderHash: "0x",
    parentOrderHash: bytes32(0),
    subOrderIndex: 0
});
bOrder.orderHash = bOrder.getOrderHash(address(transactionProcessor));
// Sign message using private key
(uint8 bV, bytes32 bR, bytes32 bS) = vm.sign(PRIVATE_KEY_B, bOrder.orderHash);
bOrder.base.signature = abi.encodePacked(bR, bS, bV);

Trade memory trade = Trade({
    partyAOrder: aOrder,
    partyBOrder: bOrder,
    actualCollateral: 1,
    actualSynthetic: 1,
    actualAFee: 0,
    actualBFee: 0
});
bytes memory message = abi.encode(trade);

bytes memory txBytes = getMessage(TYPE_TRADE, message);

bytes[] memory txs = new bytes[](1);
txs[0] = txBytes;
executeMultiTransactions(txs);
AccountPosition memory accountPosition =
↪  positionManager.getAccountPositionByUID(deployerUid);
console.log("position.ColleteralBalance", accountPosition.collateralBalance); //
↪  999999
console.log("position.assets[0].balance", accountPosition.assets[0].balance); // 1


AccountPosition memory accountPositionB =
↪  positionManager.getAccountPositionByUID(bOwnerUid);
```

```
    console.log("position.ColleteralBalance", accountPositionB.collateralBalance); // 1
    console.log("position.assets[0].balance", accountPositionB.assets[0].balance); // -1
}
```

Recommendation: Since price comparisons do not perform division, meaning no rounding occurs, legitimate orders will not be rejected due to price rounding. Therefore, it is recommended not to round prices during comparison.

EdgeX: Fixed in commit 18897c8.

Cantina Managed: Verified.

3.2.8 Liquidation has no price limit

Severity: Medium Risk

Context: PositionManager.sol#L304-L307

Description: When liquidating, the liquidator can use any price order to liquidate the risk account and must pass the checks in `_checkValidTransition()`.

```
function _checkValidTransition(
    AccountPosition memory initPosition,
    AccountPosition memory updatedPosition,
    bool isLiquidation
) internal view {
    (int256 updatedTotalValue, int256 updatedTotalRisk) =
    ↪  getAccountPositionStatus(updatedPosition);
    if (updatedTotalRisk <= updatedTotalValue * FXP_32_ONE) {
        // Position is healthy.
        // For liquidation: check sign change to prevent over-closing.
        // For normal trades: no restriction, user can flip position direction.
        if (isLiquidation) {
            _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, true);
        }
        return;
    }
    if (updatedTotalRisk == 0) {
        return;
    }
    (int256 initTotalValue, int256 initTotalRisk) =
    ↪  getAccountPositionStatus(initPosition);

    // High risk - full check: position reduction only, no position increase, no new
    ↪  asset IDs.
    _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, false);

    // total_value / total_risk must not decrease.
    // tv0 / tr0 <= tv1 / tr1 iff tv0 * tr1 <= tv1 * tr0.
    // tv is 96 bit, tr is 128 bit, so tv*tr fits in 224 bits.
    // Since tv can be negative, adding SIGNED_COMPARISON_OFFSET to each side.
    if (
        (SIGNED_COMPARISON_OFFSET + initTotalValue * updatedTotalRisk)
            > (SIGNED_COMPARISON_OFFSET + updatedTotalValue * initTotalRisk)
    ) {
        revert("PM: val/risk dec");
    }
```

Based on the checks in `_checkValidTransition()`, the liquidator must either bring the post-liquidation health factor (`tv/tr`) back above 1 or cannot decrease the health factor. Therefore, the most profitable approach for the liquidator is to keep the health factor unchanged after liquidation. Assume a liquidator provides x collateral to liquidate y assets, with the liquidator's price being p0 = x / y and the oracle price being p1. The calculation is as follows:

```
tv0 / tr0 <= tv1 / tr1
```

```
tv0 + x - p1 * y = tv1

tr0 - y * p1 * riskFactor = tr1

x = p0 * y

tv0 + p0 * y - p1 * y = tv1

tr0 - y * p1 * riskFactor = tr1

tv0 / tr0 <= tv1 / tr1 = (tv0 + p0 * y - p1 * y) / (tr0 - y * p1 * riskFactor)

tv0 * (tr0 - y * p1 * riskFactor) <= (tv0 + p0 * y - p1 * y) * tr0

tv0 * tr0 - tv0 * y * p1 * riskFactor <= tr0 * tv0 + p0 * y * tr0 - p1 * y * tr0

- tv0 * y * p1 * riskFactor <= p0 * y * tr0 - p1 * y * tr0

- tv0 * p1 * riskFactor <= p0 * tr0 - p1 * tr0

p1 * tv0 * riskFactor >= (p1 - p0) * tr0

tv0 / tr0 >= (p1 - p0) / (p1 * riskFactor)
```

So the check requires `(p1 - p0) / (p1 * riskFactor) ≤ tv0 / tr0`. The minimum price the liquidator can provide is `oraclePrice * (1 - riskFactor * initialHealthFactor)`.

- Consider ETH/USDT = 3000, riskFactor = 0.2.
- The user's account collateral is 6000 USDT. The user uses 27000 USDT to trade for 9 ETH.
- Total value: -21,000 + 9 * 3,000 = 6,000. Total debt: 9 * 3,000 * 0.2 = 5,400. Factor: 1.11.
- ETH/USDT drops to 2900, total value becomes -21000 + 9 * 2900 = 5100, total debt is 9 * 2900 * 0.2 = 5220, factor is 0.977. Liquidatable.
- Based on the formula above, the worst price that the liquidator can provide is calculated to be (2900 - x) / 580 <= 0.977, x >= 2333.34.
- The liquidator uses the low price of 2333.34 to liquidate 1 ETH.
- Total value: -21000 + 2333.34 + 8 * 2900 = 4533.34. Total debt: 8 * 2900 * 0.2 = 4640. factor is 0.977. Valid.
- This may result in the risk account being unable to return to healthy state.

Note that the above example applies to long positions, for short positions, liquidators may sell synthetic assets at a higher price.

Recommendation: It is recommended to set a price slippage (e.g., 5%) for liquidation, allowing liquidators to use the worse price within the slippage range to liquidate risk accounts.

EdgeX: Acknowledged.

Cantina Managed: Acknowledged.

3.3 Low Risk

3.3.1 `depositWithPermit()` allows attackers to deposit funds that user has approved into the `edgexVault`

Severity: Low Risk

Context: LoadingZone.sol#L108-L120

Description: `depositWithPermit()` uses a try-catch block to wrap the `permit()` call to prevent frontrun, but this also introduces another problem.

```
function depositWithPermit(address user, uint256 amount, uint256 deadline, uint8 v,
↪  bytes32 r, bytes32 s) external {
    require(amount > 0, "LoadingZone: amount must be greater than 0");
    require(address(edgexVault) != address(0), "LoadingZone: edgex vault not set");

    // Try permit to EdgexVault - if it fails (e.g., frontrun), continue anyway since
    ↪  allowance may already be set
    try IERC20Permit(address(depositToken)).permit(user, address(edgexVault), amount,
    ↪  deadline, v, r, s) {} catch {}

    balances[user] += amount;
    // EdgexVault pulls tokens from user
    edgexVault.depositFrom(user, address(depositToken), amount);

    emit TokenDeposited(user, amount, balances[user]);
}
```

For user A who has already approved `LoadingZone`, an attacker could provide an invalid signature to call `depositWithPermit()`. The `permit()` call will fail without reverting, and user A's approved funds will be deposited into `edgexVault`.

Recommendation: It is recommended to document this issue to inform users to avoid approving excessive funds to `LoadingZone`.

EdgeX: Fixed in commit 1db7e23.

Cantina Managed: Verified.

3.3.2 Position Information Can Be Overwritten Resulting in Corrupted State

Severity: Low Risk

Context: PositionManager.sol#L767-L785

Description: The `PositionManager.addSyntheticPosition()` function is `public` and can be called by the transaction processor or owner. It does not validate that no position already exists for the specified account and asset, so if it is invoked on an existing position, it will overwrite the `balance` and `isolatedCollateral` values with zeros, leading to a corrupted protocol state.

Fortunately, the transaction processor doesn't have any logic that calls this function, so the only way to use it to overwrite an existing position is to have control over the `owner` address. This could occur either accidentally or due to a key compromise (although in the latter case, there are likely multiple more severe consequences).

Recommendation: One option is to add the missing validation for the non-existence of the position being added (including validating the that `assetId` provided is not one of the two invalid values). A better option may be to simply make this function `internal` and ensure that all calling contexts perform appropriate verification (which they mostly already do, except for inadequate `assetId` verification in `getOrCreatePosition()` as described in another finding).

EdgeX: Fixed in commit 3b76dd22.

Cantina Managed: Fix verified.

3.3.3 Additional Verification Check Recommendations

Severity: Low Risk

Context: AccountManager.sol#L75, AccountManager.sol#L107, PositionManager.sol#L759

Description: The codebase has a highly modular structure, and many functions are called not only from multiple other functions or contracts, but can also be called directly by the `owner` of the contact they are defined in (it is unclear whether the `owner` value differs for different contracts or will be a single globally-used address). Calling contexts often have inconsistent validation of input parameters, and for direct `owner` calls only the validation in a specific function is applied. This finding aggregates a number of input validation enhancements that are recommended to prevent unintentional state corruption.

- AccountManager.updateCollateralBalance(): When called via the permission manager, the specified account id is only sometimes verified to be registered beforehand, and there's no registration check if called directly by the `owner`. It would be safer to add an account registration check within this function to ensure that only registered accounts may have a non-zero collateral balance.
- AccountManager.addAsset(): Verify that `assetId != 0 && assetId != NO_SYNTHETIC_DELTA_ASSET_ID`, that the account is registered, and the asset is not the collateral asset.
- GeneralConfig.getRiskFactor(): Based on naming and usage in PositionManager.sol, it appears that `absBalance` is intended to be a strictly non-negative number here. Consider either verifying that to be the case, or passing it as an unsigned type like `uint256`.
- PositionManager.getOrCreatePosition(): Verify that

  [] assetId != 0 && assetId != NO_SYNTHETIC_DELTA_ASSET_ID

  that the account is registered, and the asset is not the collateral asset.
- PositionManager.forceSettlePosition(): Verify that `collateralDelta` is non-negative (or pass it as a `uint256`).

Recommendation: Add the recommended validation checks. If in any case there is a desire to allow the `owner` to bypass checks, then the checks should still be enforced for non-`owner` callers.

> **Note:** this finding should not be considered exhaustive of all locations that additional verification would be beneficial as there was not enough time in the engagement to review the entire scope in full detail. It is recommended that the project team perform their own review of all contract entrypoints to assess verification sufficiency.

> **Verification Note:** Fixed by removing `owner` access to call most functions directly. Careful testing of all flows through the transaction processor is recommended to ensure that end-to-end validation is always sufficient.

EdgeX: Fixed in commit 48c138cd.

Cantina Managed: Fix verified.

3.3.4 Positions Can Exceed Value and Risk Bounds In Extreme Scenarios, Blocking Operations

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `PositionManager.getAccountPositionStatus()` performs boundary checks on the position's total value and total risk:

```
(totalValueRep, totalRiskRep) = _getAccountPositionStatusInner(position);
int256 totalValueLowerBoundRep = TOTAL_VALUE_LOWER_BOUND * FXP_32_ONE;
int256 totalValueUpperBoundRep = TOTAL_VALUE_UPPER_BOUND * FXP_32_ONE;
if (totalValueRep < totalValueLowerBoundRep || totalValueRep > totalValueUpperBoundRep) {
    revert("PM: val range");
}
int256 totalRiskUpperBoundRep = TOTAL_RISK_UPPER_BOUND * FXP_32_ONE * FXP_32_ONE;
if (totalRiskRep >= totalRiskUpperBoundRep) {
    revert("PM: risk range");
}
```

This function is used in multiple places, including checking whether a position is risky, so if these checks are violated many operations (including liquidations) will be blocked. It is theoretically possible that a position could reach such a state without any explicit state transition that would be caught by `_checkValidTransition()` (which would block e.g. a trade that opened a too-large or too-small position). For example, this could happen if the oracle price increased significantly. This is unlikely in practice due to the large magnitudes of the boundary values:

```
int256 constant TOTAL_VALUE_UPPER_BOUND = 2 ** 63;
int256 constant TOTAL_VALUE_LOWER_BOUND = -(2 ** 63);

int256 constant TOTAL_RISK_UPPER_BOUND = 2 ** 64;
```

Recommendation: No action is deemed to be strictly necessary, but one option is to move these checks to only be done within `_checkValidTransition()` itself and only for the updated position (so a position that was already in violation of the checks could be "fixed").

EdgeX: Acknowledged, no code changes planned.

1. Off-chain position size limits validations are in place to prevent reaching these bounds under normal operations. 2. The extremely large boundary values (2^63 for value, 2^64 for risk) make this scenario highly improbable in practice.

Cantina Managed: Acknowledged.

3.3.5 Rounding direction in DelistSettle is incorrect

Severity: Low Risk

Context: DelistSettleTransLib.sol#L34

Description: In DelistSettle, when `syntheticBalance` is less than 0, the truncating division when calculating `collateralDelta` causes `collateralDelta` to be larger (e.g., -2.9 rounds down to -2), which is not beneficial for the protocol.

Recommendation:

```
- int256 collateralDelta = (syntheticBalance * price) / FXP_32_ONE;
+ int256 collateralDelta = syntheticBalance > 0 ? (syntheticBalance * price) / FXP_32_ONE
↪  : (syntheticBalance * price - FXP_32_ONE + 1) / FXP_32_ONE;
```

EdgeX: Fixed in commit f0d8d82.

Cantina Managed: Verified.

3.3.6 Withdrawal structure lacks timestamps

Severity: Low Risk

Context: WithdrawalTransLib.sol#L13-L20

Description: The Withdrawal structure lacks a timestamp field. Since Withdrawal is a time-sensitive operation-for example, if a user signs a Withdrawal operation but it fails due to insufficient balance-the signature should expire after a short period. Otherwise, the signature could be abused by attacker.

```
struct Withdrawal {
    uint64 from; // Withdraw request sender account ID
    address toAddress; // Withdraw recipient address
    int256 amount; // Withdraw  amount
    uint256 nonce;
    bytes signature; // Signature of the withdrawal
    address signer; // Signer address
}
```

Recommendation: It is recommended to add a timestamp field to the withdrawal structure.

EdgeX: Fixed in commit 5eda036.

Cantina Managed: Verified.

3.3.7 Account status has no buffer between healthy and liquidatable

Severity: Low Risk

Context: PositionManager.sol#L277-L299

Description: There is no buffer between account safety and risk, an account can increase its debt to the limit, after which the account becomes liquidated if the debt appreciates even slightly.

```
function _checkValidTransition(
    AccountPosition memory initPosition,
    AccountPosition memory updatedPosition,
    bool isLiquidation
) internal view {
    (int256 updatedTotalValue, int256 updatedTotalRisk) =
    ↪  getAccountPositionStatus(updatedPosition);
    if (updatedTotalRisk <= updatedTotalValue * FXP_32_ONE) {
        // Position is healthy.
        // For liquidation: check sign change to prevent over-closing.
        // For normal trades: no restriction, user can flip position direction.
        if (isLiquidation) {
            _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, true);
        }
        return;
    }
    if (updatedTotalRisk == 0) {
        return;
    }
    (int256 initTotalValue, int256 initTotalRisk) =
    ↪  getAccountPositionStatus(initPosition);

    // High risk - full check: position reduction only, no position increase, no new
    ↪  asset IDs.
    _checkSmallerInSyntheticHoldings(initPosition, updatedPosition, false);
```

Recommendation: It is recommended to set a buffer between account status being healthy and being liquidatable.

EdgeX: Acknowledged. The protocol already implements appropriate safeguards through off-chain controls. Users cannot open new positions when their margin falls between the Initial Margin Requirement (IMR) and Maintenance Margin Requirement (MMR) - liquidation only occurs at MMR. Since all transactions are user-initiated and processed off-chain before submission, this design provides sufficient protection without requiring an on-chain buffer.

Cantina Managed: Acknowledged.

3.4 Gas Optimization

3.4.1 Gas Optimizations

Severity: Gas Optimization

Context: foundry.toml#L13, PositionManager.sol#L208, RegisterAccountTransLib.sol#L26-L34, Uint64Set.sol#L80-L87

Description:

1. AccountManager.sol#L16: It's an odd pattern pattern to mirror the mapping key in the data structure--it should be an easy gas win to not store the account id, as it needs to be known to look up the data in the first place. Not a huge optimization though since `accountId` will pack with `ownerAddress` so the same number of slots are used.

2. AccountManager.sol#L79-L80: `newValue` is unnecessary, could just emit `amount`:

   [] emit CollateralChanged(account, preValue, amount);

3. MutltiTransactionProcessor.sol#L102-L111: Move the checks for deprecated transaction types to the end of the `if...else` chain to avoid doing them for non-deprecated transaction types (optimizes for correct/standard usage which should be more frequent).

   Since the error message is the same for all three deprecated transactions, they could be further combined into a single conditional to benefit from short circuit evaluation:

[] } else if (txType == TYPE_FORCED_WITHDRAWAL || txType == TYPE_FORCED_TRADE || txType == TYPE_CONDITIONAL_TRANSFER) {

4. PositionManager.sol#L208: Rather than creating another in-memory `AccountPosition` struct, since `isEmptyPosition()` already verifies that `collateralBalance == 0` and the asset list is empty, this line could be replaced with a simple assignment of the account id:

```
acc.accountId = accountId;
```

5. PositionManager.sol#L634-L639: Cache the value of `_syntheticPositionKey(accountId, assetId)` in a stack variable instead of recomputing it.
6. RegisterAccountTransLib.sol#L26-L34: This array copying operation could be eliminated by using `SignerWithPermissions` instead of `SignerInit` in the `RegisterAccount` struct (the `SignerInit` struct could be eliminated in this case).
7. Uint64Set.sol#L80-L87: The creation of a dedicated return variable isn't necessary here as `_values()` already returns a memory object. This function body can be simplified to:

   [] return _values(set._inner);

8. ***(Out of Scope)*** foundry.toml#L13: Could consider a higher value for `optimizer_runs`, which should improve runtime performance (at the cost of potentially larger bytecode size).

Recommendation: Implement the optimizations described above.

> **Verification Note:** The fix commit provided by the client addresses items 1, 5, and 6. Items 2, 3, 4, and 7 were also addressed in another commit. This leaves only the out-of-scope item 8 unaddressed.

EdgeX: Fixed in commit 659e3ffe.

Cantina Managed: Fix verified.

3.5 Informational

3.5.1 Minor Issues: Redundancies, Event Issues, Code Clarity, etc..

Severity: Informational

Context: *(See each case below)*

Redundancies and Unused Code Constructs: A number of functions always return `true` and revert on any error, making their return values redundant and increasing gas costs. See list below. These functions can be changed to not return any value at all, and any checks of their return values can be removed.

- `EIP712VerificationLib.verifyWithdrawalSignature()`.
- `EIP712VerificationLib.verifySetMarginPreferenceSignature()`.
- `EIP712VerificationLib.verifyAddIsolatedMarginSignature()`.
- `EIP712VerificationLib.verifyRemoveIsolatedMarginSignature()`.
- `EIP712VerificationLib.verifySetSignerPermissionsSignature()`.
- `EIP712VerificationLib.verifyAddSignersWithPermissionsSignature()`.
- `EIP712VerificationLib.verifyTransferSignature()`.
- `EIP712VerificationLib.verifyRegisterAccountSignature()`.
- `EIP712VerificationLib.verifyTwapOrderSignature()`.
- `EIP712VerificationLib.verifyLimitOrderSignature()`.
- `EIP712VerificationLib._enforcePermissionsAndNonce()` (transitively because it returns the same value as `_enforcePermissionsAndNonceInternal()`).
- `EIP712VerificationLib._enforceOrderPermissionsAndNonce()` (transitively because it returns the same value as `_enforcePermissionsAndNonceInternal()`).

- `EIP712VerificationLib._enforcePermissionsAndNonceInternal()`.
- `LoadingZone.processDeposit()`.
- `LoadingZone.processWithdraw()`.

- AccountManager.sol#L57-L60: The `onlyTransactionProcessor()` modifier is unused and can be removed.
- LimitOrderLib.sol#L109-L111: Never evaluates to `true`--the check on line 91 will cause a revert first. Returning is also likely the wrong behavior. Simply remove this check since there's already a better one.
- LimitOrderLib.sol#L226: As a `uint256`, `subOrderIndex` is always `>= 0` so the first part of this check is unnecessary and can be removed.
- LoadingZone.sol#L34: Unused event (`TokenWithdrawn`)--should this be emitted somewhere? If not, it can be removed.
- MultiTransactionProcessor.sol#L89-L92: The `require` statement on line 77 enforces that `txs.length > 0`, and since `len == txs.length`, `processedIndex` is always set to `int64(int256(len) - 1)`. The `MultiTransactionsProcessed` event is already emitting `len` as its second argument, so in effect it always emits `(len - 1, len)`. Assuming this function is otherwise logically correct, `MultiTransactionsProcessed` should be simplified to only emit `len` as emitting both `len` and `len - 1` is redundant.
- PositionManager.sol#L86, PositionManager.sol#L99-L100: Overwriting the margin mode in the `packed` values on these lines should be unnecessary, the margin mode of a `syntheticPositions` entry should always match the account's margin mode preference for the relevant asset. Note also that doing so is inconsistent with the implementations of `_loadCrossAccountPosition()` and `_loadIsolatedAccountPosition()`.
- PositionManager.sol#L305-L306: The use of `SIGNED_COMPARISON_OFFSET` is not necessary. Adding the same quantity to both sides of an inequality does not change its truth value, nor is it required for the comparison operator to work correctly.
- PositionManager.sol#L467-L481: These two functions have the same arguments, implementation, and return value. Either one of them should be modified, or one can be deleted. Note that `getIsolatedAccountPositionByAssetId()` is missing from the `IPositionManager` interface, is inconsistent with other functions that use the UID naming terminology, and is only used in tests (which can use `getIsolatedAccountPositionByUID()` instead), so removing it seems like the most logical option.
- PositionManager.sol#L483-L517: `incrementCollateralOnly()` and `incrementCollateralOnlyForFeeAccount()` share a lot code, they could be refactored to reduce duplication.
- PositionManager.sol#L761: This check cannot fail and can be removed.
- SignaturesLib.sol: Other than being imported in a few files, the `SignaturesLib` library isn't actually used in this repository. Notably, `EIP712Lib` uses OpenZeppelin's `SignatureChecker` instead. This library can be removed (or, if it was intended to be used, then add that missing logic).
- TransactionProcessor.sol#L121-L123: An equivalent check is also performed in `WithdrawTransLib.process()`; this redundant check should be removed to be consistent with the pattern of all validation occurring in `process()` functions.

Events:

- PositionManager.sol#L496-L498: It's not really necessary to emit this event, as the `accountManager.updateCollateralBalance()` call already emits an event with data that includes what is emitted here. Since this is the only usage of the `AccountCollateralUpdated` event, if this emission is removed, the event can also be eliminated.
- PositionManager.sol#L620-L624: This event emission doesn't make sense for two reasons:

1. `existing` is a storage pointer to `syntheticPositions[key]`, which was just deleted on line 618 above. So `existing` now points to empty storage, and assigning `assets[0] = existing;` does

nothing--`assets[0]` is still an empty `PositionAsset` struct. Possibly the type of `existing` needs to be corrected to `PositionAsset memory`, although then its `isolatedCollateral` value will be wrong in some edge cases and needs further code corrections.

2. Normally, when an account position is updated, the `AccountPositionUpdated` event contains the full position. Here, it will not. For example, if the user has both a cross collateral position and an isolated position, and the isolated position is forcibly settled, the assets associated with the cross collateral position will not appear in the event data.

- PositionManager.sol#L788: State change without event emission.
- SignerManager.sol#L175: State change without event emission.

Naming:

- DepositTransLib.sol#L26: Naming this parameter `positionManager` instead of `position` would be clearer and more consistent with the naming style used elsewhere (e.g. see `accountManager` on the next line).
- PositionManager.sol#L759: The function name `getOrCreatePosition()` implies that a position should be returned by this function, but this is not done. Either modify it to return a position value, or adjust the name to something less likely to cause confusion (e.g. `createPositionIfNecessary()`).
- TradeTransLib.sol#L31: Rename this argument to `positionManager`.
- TransactionProcessor.sol#L48: Use `positionManager` instead of `position`.
- TransactionProcessor.sol#L51: Use `orderManager` instead of `order`.
- TransactionProcessor.sol#L53-L54: Use `accountManager` instead of `account` and `signerManager` instead of `signer`.
- TransactionProcessor.sol#L286-289: Rename this function to `setPositionManger()`, the argument to `_positionManager`, and the event to `PositionManagerSet`.
- TransactionProcessor.sol#L301-L304: Rename this function to `setOrderManager()`, the argument to `_orderManager`, and the event to `OrderManagerSet`.
- TransactionProcessor.sol#L311-L314: Rename this function to `setAccountManager()`, the argument to `_accountManager`, and the event to `AccountManagerSet`.
- TransactionProcessor.sol#L320: Rename the argument to `_signerManager`.

Comments and NatSpec:

- PositionManager.sol#L67-L69: This comment seems to be incorrect and/or out-of-place. The function it would most logically apply to, `_loadAccountPosition()` immediately below, is not invoked during execution of `updatePositionInDict()`. Consider deleting it, modifying it, or moving it where it is meant to be.
- OrderManager.sol#L37: This comment is too vague to provide any useful information, it should be removed.

Interfaces:

- IAccountManager.sol: This interface is missing the following functions defined on the actual `AccountManager` contract: `setPositionAdmin()`, `setTransactionProcessor()`, `positionManager()` and `transactionProcessor()`.
- PositionManager.sol#L213-L215: This function doesn't appear in the `IPositionManager` interface and would be easy for a caller to compute--it should probably be made `internal` (if not, add it to the interface).

Best Practices: All files with imports: it is considered a best practice to use named imports in Solidity, e.g. `import { X } from "x.sol";` since multiple importable symbols may be defined in any given file.

Only use signed integers if it is necessary to allow negative values. This makes intent clearer and can reduce the need for validation.

Custom errors versus error strings: the code exclusively uses error strings in `require` and `revert` statements. Custom errors use less gas and provide a convenient and standardized format for emitting structured data to help with off-chain error analysis.

Enums are more self-documenting and type-safe than integer constants. The Solidity compiler will also automatically use the smallest necessary integer container for an enumeration, so they do not increase gas costs. Consider using an `enum` for the following collections of constants:

1. Constants.sol#L29-L30: margin modes.
2. Constants.sol#L34-L52: transaction types.

Miscellaneous Issues:

- EIP712VerificationLib.sol#L647: This line is abusing the fact that `size` happens to be equal to the EVM word size--the same code copy-pasted to hash an array of objects smaller or larger than 32 bytes would be incorrect (assuming `size` gets assigned to the appropriate value in the hypothetical case). It would be cleaner to just hardcode the word size when accounting for the word used to store the length of `buffer`:

  [] mstore(add(add(buffer, 32), mul(i, size)), slot)

  Or one could use a separate constant:

  [] uint256 constant EVM_WORD_SIZE_BYTES = 32; //... mstore(add(add(buffer, EVM_WORD_SIZE_-BYTES), mul(i, size)), slot)

- Funding.sol#L96: Consider a configurable max age parameter instead of hardcoding `5 minutes`.
- PositionManager.sol#L753: Since `assetId` should never be zero if the `assets` list has a non-zero length, it may be better to throw an error here as this would represent a serious invariant violation.

Recommendation: Address each item either as suggested or in a more appropriate way to improve the maintainability, clarity, and robustness of the codebase.

Verification Note: Most items have been fixed. The following items were partially fixed:

- MultiTransactionProcessor.sol#L89-L92: Partially fixed, the ternary computation was removed but the event is still emitting redundant information.
- SignaturesLib.sol: File not deleted, but import removed in `DepositTransLib.sol`, `OraclePriceTickTransLib.sol`, and `WithdrawalTransLib.sol`.

The suggestions in the Best Practices section were not adopted; it is still recommended to consider these going forward, especially if any new codebases are started.

EdgeX: Fixed in commit ae96731d.

Cantina Managed: Fix verified.

3.5.2 EIP-712 Compliance

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Most of the structs used to passed transaction parameters include a field for an EIP-712 signature, for example:

```
struct RegisterAccount {
    uint64 accountId;
    address owner; // owner address
    SignerInit[] signers; // non-owner signers and their permissions
    bytes signature; // EIP-712 signature authorizing registration
}
```

Because it would be infeasible to compute to compute a signature over data that depends on the signature itself, the structs are hashed without the signature field and it is omitted from the typehashes, for example:

```
bytes32 public constant REGISTER_ACCOUNT_TYPEHASH = keccak256(
    "RegisterAccount(uint64 accountId,address owner,SignerInit[]
    ↪  signers)SignerInit(address signer,uint256 permissions)"
);
```

This technically violates EIP-712, as all struct data members should be included in the typehash. This could cause issues with external tooling or libraries, necessitating custom logic or leading to bugs.

Recommendation: Consider conforming to EIP-712, either by passing signatures separately from parameter structs, or using a nested schema (see example below).

```
struct RegisterAccountParams {
    uint64 accountId;
    address owner; // owner address
    SignerInit[] signers; // non-owner signers and their permissions
}

struct RegisterAccount {
    RegisterAccountParams params;
    bytes signature; // signature is for the params member defined above
}
```

EdgeX: Fixed in commit 3a0f9a7d.

Cantina Managed: Fix verified.